FWD Mini-Project Report

A project on an Audio Streaming Website

By: C067 Parth Kotecha, C077 Abhyudit Agrawal, C086 Jash Ghelani

Our Aim:

Our innovative audio streaming website is a testament to our team's dedication and passion for creating a platform that brings the joy of music to everyone. This project was conceived with the vision of providing music enthusiasts a seamless and immersive listening experience, transcending boundaries and genres.

HTML:

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Audio Streaming Website</title>
    <link rel="stylesheet" href="styles.css">
<script type="text/javascript" src="https://gc.kis.v2.scr.kaspersky-
labs.com/FD126C42-EBFA-4E12-B309-BB3FDD723AC1/main.js?"
nonce="c5068191ecf57a5f1cd40dbe140ddb62" charset="UTF-8"></script><link
rel="stylesheet" crossorigin="anonymous"
href="https://gc.kis.v2.scr.kaspersky-labs.com/E3E8934C-235A-4B0E-825A-
35A08381A191/abn/main.css? "/></head>
<body>
    <header>
        <h1>Audio Streaming Website</h1>
        <input type="text" id="search" placeholder="Search for music...">
    </header>
    <main>
        <div class="music-list">
            <!-- Music tracks will be dynamically loaded here -->
        </div>
    </main>
    <script src="script.js"></script>
</body>

<section class="about-us">
```

```
    <h2>About Us</h2>
    <p>We are the creators of this music streaming website. <br><br>Meet the
team:</p>
    <ul>
        <li>Parth Kotecha (C067)</li>
        <li>Jash Ghelani (C086)</li>
        <li>Abhyudit Agrawal (C077)</li>
    </ul>
</section>
</body>
</html>
```

The used HTML code represents the structure of an Audio Streaming Website. Here is an analysis of the tags and technologies utilized within the code:

1. Document Type Declaration: The code starts with the document type declaration defining it as an HTML5 document.

```
<!DOCTYPE html>
```

2. Language and Charset Declaration: The language of the document is set to English, and the character set is specified as UTF-8.

```
<html lang="en">
<head>
  <meta charset="UTF-8">
```

3. Viewport Configuration: The viewport is set to control the layout on different devices.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

4. Title Tag: Sets the title of the web page displayed on the browser's title bar.

```
<title>Audio Streaming Website</title>
```

5. Linking CSS: External stylesheet 'styles.css' is linked to stylize the webpage.

```
<link rel="stylesheet" href="styles.css">
```

6. JavaScript File Inclusion: The script file 'script.js' is included at the end of the body, suggesting it holds interactive functionality for the website.

```
<script src="script.js"></script>
```

7. Header Section: Contains the main heading and a search input for music search functionality.

```
<header>
    <h1>Audio Streaming Website</h1>
    <input type="text" id="search" placeholder="Search for music...">
</header>
```

8. Main Content Section: Includes a section where the music list will be dynamically loaded.

```
<main>
    <div class="music-list">
        <!-- Music tracks will be dynamically loaded here -->
    </div>
</main>
```

9. About Us Section: Provides information about the creators of the website.

```
<section class="about-us">
  <h2>About Us</h2>
  <!-- Information about the creators -->
</section>
```

10. List of Team Members: Lists the names and codes of the creators.

```
<ul>
  <li>Parth Kotecha (C067)</li>
  <li>Jash Ghelani (C086)</li>
  <li>Abhyudit Agrawal (C077)</li>
</ul>
```

11. External Script Inclusion: An external script seems to be included within a `<script>` tag in the `<head>`, but the URL appears to be extremely long. This is injected code. It's recommended to review and validate this script source for security reasons.

12. Link Tag with an Unusual URL: Another link tag in the `<head>` includes a URL that seems to be highly unusual and might not be intended for standard web development. It's advised to review and potentially remove this link if it's not essential for the website's functionality.

CSS:

Code:

```
/* Reset and global styles */
body {
    font-family: 'Helvetica Neue', Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #000; /* Black background */
    color: #454545; /* Chestnut text color */
}

/* Header styles */
header {
    background-color: #840f0f; /* Chestnut header background */
    padding: 20px 0;
    text-align: center;
}

h1 {
    margin: 0;
    font-size: 36px; /* Larger font for the title */
    color: #fff; /* White title text color */
}

h2 {
    margin: 0;
    font-size: 18px;
    color: #fff;
}

/* Search bar styles */
input[type="text"] {
    padding: 12px;
    width: 80%;
    max-width: 400px; /* Limit the width of the search bar */
    border: 2px solid #000; /* Black border */
    border-radius: 25px; /* Rounded corners for the search bar */
    font-size: 16px;
    background-color: #454545; /* Chestnut background color */
    color: #fff; /* White text color */
    outline: none; /* Remove the default blue outline on focus */
    transition: border-color 0.3s; /* Add a transition for border color */
}

/* Change border color on focus */
input[type="text"]:focus {
    border-color: #000; /* Black border color on focus */
```

```
}


/* Main content styles */
main {
    padding: 20px;
}

/* Music list styles */
.music-list {
    display: grid; /* Use grid layout for music items */
    grid-template-columns: repeat(auto-fill, minmax(250px, 1fr)); /*
Responsive grid columns */
    gap: 20px;
}

.music-item {
    padding: 20px;
    background-color: #840f0f; /* Chestnut background for music items */
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
    border-radius: 8px;
    transition: transform 0.2s; /* Add a subtle hover effect */
}

.music-item img {
    max-width: 100%; /* Ensure images fit within their containers */
    height: auto; /* Maintain image aspect ratio */
    display: block; /* Remove any extra space below the image */
    margin: 10px 0; /* Add some space between the image and title */
}

.music-item:hover {
    transform: scale(1.05); /* Enlarge the card on hover */
}

/* Audio player styles */
audio {
    width: 100%;
    margin-top: 10px;
}

/* About us section styles */
.about-us {
    background-color: #000; /* Dark background for the about section */
    color: #454545; /* Chestnut text color */
    padding: 20px;
    text-align: center;
}
```

```css
.about-us h2 {
    margin-top: 0;
    font-size: 24px; /* Larger font for the section title */
    color: #fff; /* White title text color */
}

.about-us ul {
    list-style: none;
    padding: 0;
}

.about-us li {
    font-size: 18px;
    margin: 10px 0;
}
```

Our CSS explained:

Global Styles:

The global styles defined for the entire page set the foundational appearance for various elements. The `body` selector specifies the font family, background color, and text color used throughout the site.

```
body {
   font-family: 'Helvetica Neue', Arial, sans-serif;
   margin: 0;
   padding: 0;
   background-color: #000; /* Black background */
   color: #454545; /* Chestnut text color */
}
```

Header Styles:

The `header` section contains styles for the website's header area. It defines the background color, padding, and center alignment of content within the header.

```
header {
    background-color: #840f0f; /* Chestnut header background */
    padding: 20px 0;
    text-align: center;
}
```

Search Bar Styles:

Styling for the search bar is targeted using an attribute selector. It defines properties like padding, width, border, border-radius, background color, text color, and transitions for a smooth border color change on focus.

```
input[type="text"] {
    /* Search bar styles */
    /* Properties such as padding, width, border, etc. */
}

input[type="text"]:focus {
    /* Change border color on focus */
    /* ... */
}
```

Main Content and Music List Styles:

The `main` section contains padding to space out the main content. Within the `.music-list`, grid layout properties are defined for responsive music items display.

```
main {
    padding: 20px;
}

.music-list {
```

```
    display: grid;
    /* Grid column settings for responsive display */
    /* ... */
}

.music-item {
    /* Styles for individual music items */
    /* ... */
}
```

Audio Player Styles:

Styles for the `audio` element specify its width and margin to create appropriate spacing within the webpage.

```
audio {
    width: 100%;
    margin-top: 10px;
}
```

About Us Section Styles:

The `.about-us` section defines the appearance of the 'About Us' part of the website. It includes background color, text color, padding, and text alignment.

```
.about-us {
    background-color: #000; /* Dark background for the about section */
    color: #454545; /* Chestnut text color */
    padding: 20px;
    text-align: center;
}
```

Within the 'About Us' section, specific styles are applied to the `<h2>` element for the title and the `<ul>` and `<li>` elements for team member details.

Summary:

The CSS code provides a structured set of styles for various sections and elements of an Audio Streaming Website. It aims to create a visually appealing and functional layout by adjusting fonts, colors, layout structures, and interactive elements like the search bar and music items. The styles cater to both the aesthetic presentation and user interaction within different sections of the webpage, making it responsive and engaging for users.

JavaScript:

Code:

```
const musicData = [
    { title: 'Happier', source: 'song1.mp3', image: 'song1.jpg' },
    { title: 'Wow.', source: 'song2.mp3', image: 'song2.jpg' },
    { title: 'My Eyes', source: 'song3.mp3', image: 'song3.jpg' },
    { title: 'Stuck in a Dream', source: 'song4.mp3', image: 'song4.jpg' },
    { title: 'Young Dumb & Broke', source: 'song5.mp3', image: 'song5.jpg' },
    { title: 'Congratulations', source: 'song6.mp3', image: 'song6.jpg' },
    { title: 'Lifes a Mess', source: 'song7.mp3', image: 'song7.jpg' }
];


const musicList = document.querySelector('.music-list');
const searchInput = document.getElementById('search');

function displayMusicItems() {
    musicList.innerHTML = '';
    const searchTerm = searchInput.value.toLowerCase();

    musicData.forEach((music, index) => {
        if (music.title.toLowerCase().includes(searchTerm)) {
            const musicItem = document.createElement('div');
            musicItem.classList.add('music-item');
            musicItem.innerHTML = `
                <h2>${music.title}</h2>
                <img src="${music.image}" alt="${music.title} Image">
                <audio controls>
                    <source src="${music.source}" type="audio/mpeg">
                    Your browser does not support the audio element.
                </audio>
```

```
            `;
            musicList.appendChild(musicItem);
        }
    });
}

searchInput.addEventListener('input', displayMusicItems);
displayMusicItems();
```

Tags and Technologies Used in the JavaScript Code:

The JavaScript code provided focuses on handling the display of music items based on search input. It interacts with the HTML and facilitates dynamic rendering of music elements.

Array Data:

An array named `musicData` is used to store information about various music tracks, including their title, audio source, and image source. This array acts as a data source for dynamically creating music items.

const musicData = [

  // Array containing information about music tracks

];

DOM Interaction:

1. Variables Selection:

- `musicList`: Fetches the HTML element with the class `.music-list`.

- `searchInput`: Retrieves the HTML element with the ID `search`.

const musicList = document.querySelector('.music-list');

const searchInput = document.getElementById('search');

2. Displaying Music Items:

- `displayMusicItems()`: Function responsible for rendering music items based on the search input.

- On every input change within the search bar, this function is called to update the displayed music items.

- It clears the current `musicList` content and dynamically generates music items based on the search term's match with the music title.

```
function displayMusicItems() {
    // Clears the existing music list content
    // Fetches the search term from the input
    // Loops through musicData and generates music items dynamically
}
```

Generation of Music Items:

Within the `displayMusicItems()` function, the code dynamically creates HTML elements (using `document.createElement`) for each music track from the `musicData` array. It appends them to the `musicList` container using `appendChild`. It leverages the data in `musicData` to populate the content for each music item.

```
musicData.forEach((music, index) => {
  if (music.title.toLowerCase().includes(searchTerm)) {
    const musicItem = document.createElement('div');
    musicItem.classList.add('music-item');
    musicItem.innerHTML = `
      <!-- HTML structure for each music item -->
    `;
    musicList.appendChild(musicItem);
  }
});
```

Event Handling:

1. Event Listener:

- `searchInput.addEventListener('input', displayMusicItems)`: Listens for input changes in the search bar and triggers the `displayMusicItems()` function to update the displayed music items accordingly.

- Immediately after setting up the event listener, the `displayMusicItems()` function is called to populate the initial music items on page load.

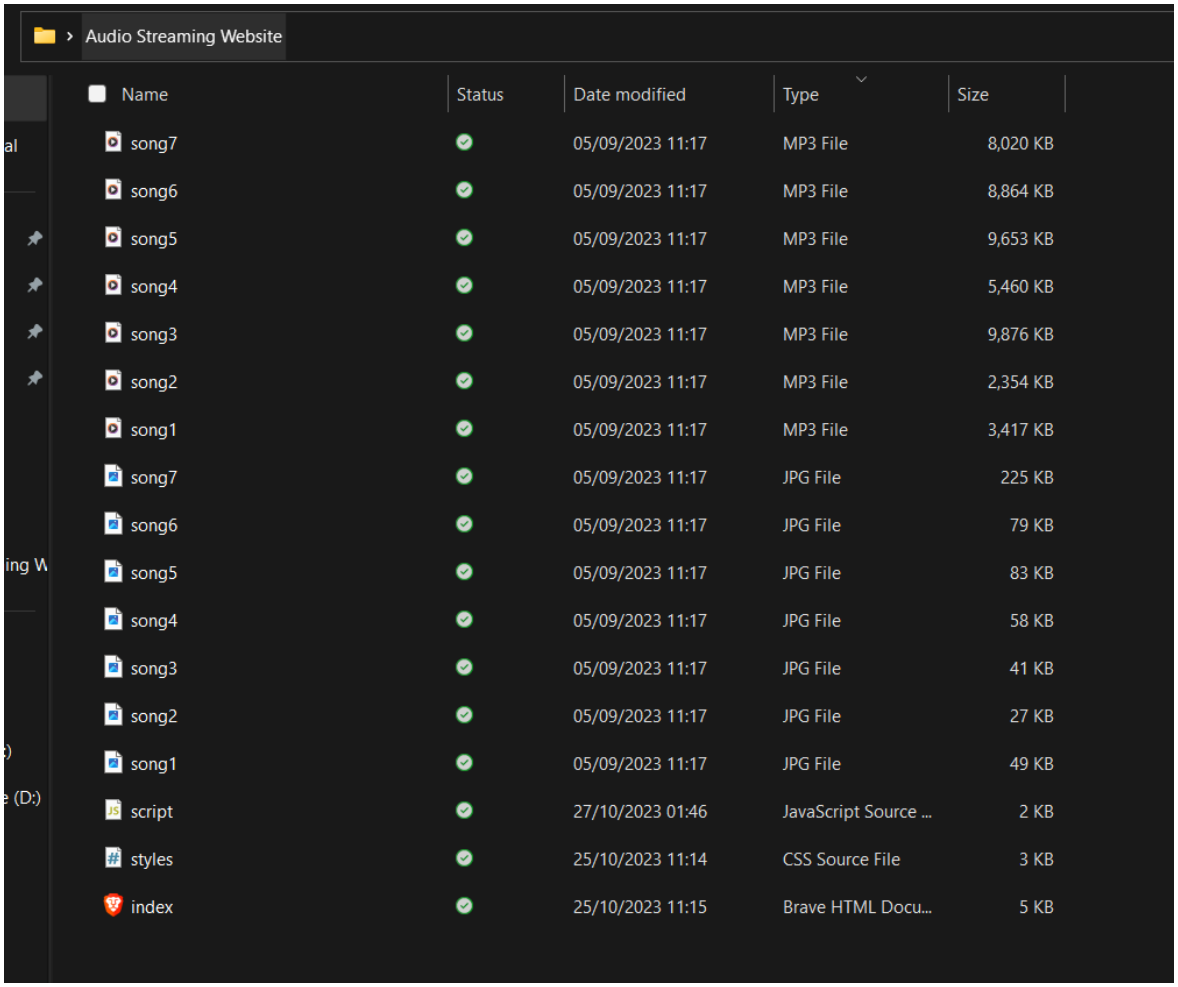
```
searchInput.addEventListener('input', displayMusicItems);

displayMusicItems(); // Initial rendering of music items
```

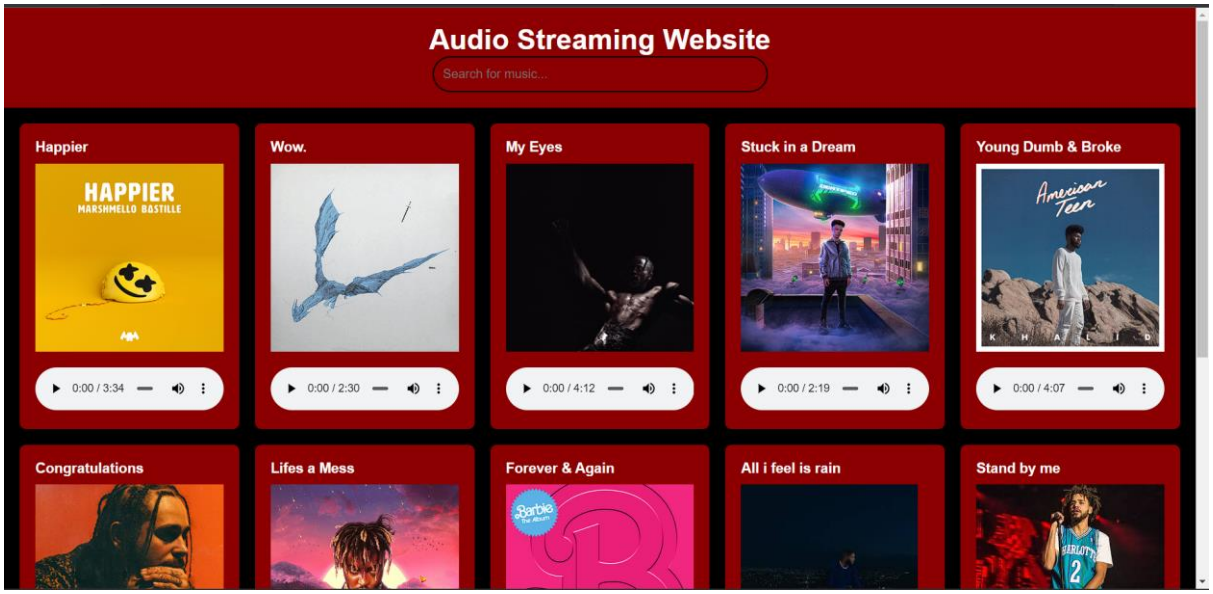
The JavaScript code efficiently interacts with the HTML structure, utilizing DOM manipulation methods to dynamically generate music items based on user input in the search bar. It ensures a responsive and interactive music display by updating content based on the search term entered by the user.

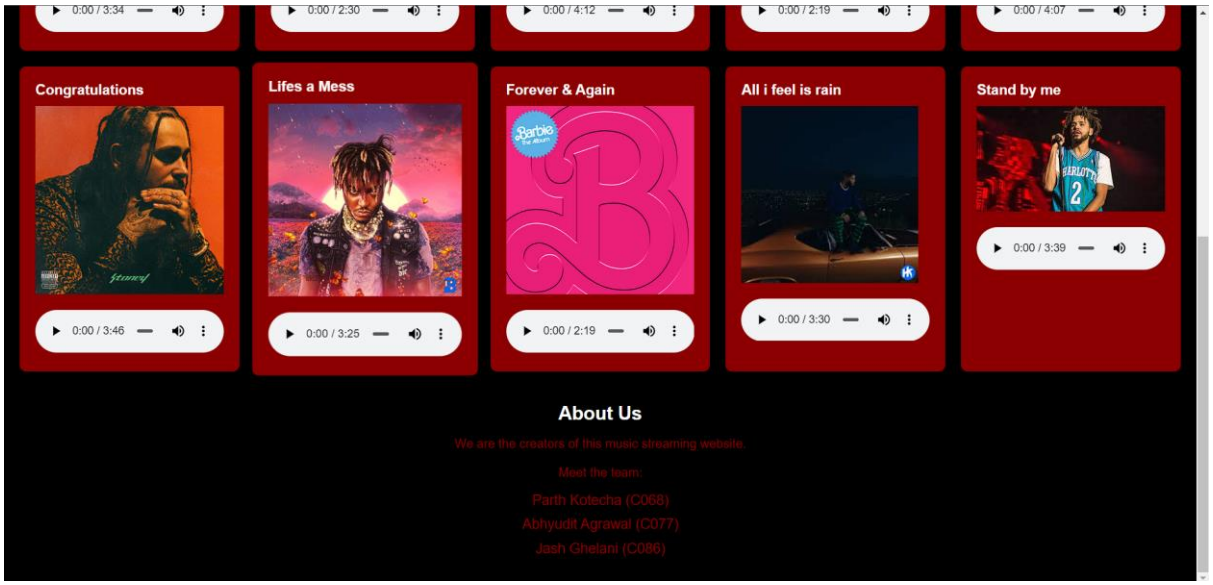
File Storage:

All data is locally stored in one folder:



Output:





Conclusion:

- Our code uses a combination of HTML, JavaScript, CSS, and related technologies to create a web page.
- It displays a list of music items, with the ability to filter and play music tracks based on a search input.
- It leverages DOM manipulation and event handling to provide an interactive user experience.